00_indice_geral

🏥 PROJETO AURA CLINIC — MANUAL DE IMPLEMENTAÇÃO PROFISSIONAL

Sistema de Gerenciamento de Clínica Médica

> **Stack:** Java 17 + Spring Boot 3.2 + MySQL | **Arquitetura:** Microsserviços | **Nível:** Intermediário

---

📚 ÍNDICE GERAL DO MANUAL

O manual está dividido em **7 partes** para facilitar a navegação:

Parte	Arquivo	Conteúdo	Seções
1	`01_arquitetura_microsservicos.md`	Visão geral, definição dos 3 microsserviços, endpoints, payloads	1–2
2	`02_banco_de_dados_estrutura.md`	Modelagem ER, SQL completo, estrutura de pastas	3–4
3	`03_padronizacao_cronograma_tarefas.md`	Convenções, GitFlow, cronograma, equipe, lista de tarefas	5–9
4	`04_comunicacao_seguranca_excecoes.md`	OpenFeign, JWT, RBAC, exceções globais, logs	10–13
5	`05_testes_swagger_docker.md`	Testes unitários/integração, Postman, Swagger, Docker	14–18
6	`06_versionamento_checklists_riscos.md`	GitFlow, PRs, checklists, riscos técnicos	19–21
7	`07_escalabilidade_boas_praticas.md`	Melhorias futuras, requisitos opcionais, boas práticas	22–23

🎯 ESCOPO DO PROJETO (BASEADO NOS DIAGRAMAS UML)

Módulos Obrigatórios

Módulo UML	Microsserviço	Porta	Banco	Ator(es)
Administrativo	`admin-service`	8081	`admin_db`	Administrador (ADM)
Agendamento	`agendamento-service`	8082	`agendamento_db`	Secretária, Médico
Atendimento	`atendimento-service`	8083	`atendimento_db`	Médico
(Suporte)	`commons`	—	—	—

Funcionalidades por Módulo

1. MÓDULO ADMINISTRATIVO (admin-service)

- Gerenciar Médicos (CRUD)
- Gerenciar Pacientes (CRUD)
- Gerenciar Especialidades (CRUD)
- Gerenciar Perfis (Usuários e permissões)
- Gerenciar Convênios (CRUD)
- Gerenciar Relatórios (relatórios gerenciais)

2. MÓDULO DE AGENDAMENTO (agendamento-service)

- Agendar Consulta (Secretária) — inclui Validar Convênio
- Remarcar Consulta (Secretária)
- Cancelar Consulta (Secretária)
- Consultar Agenda (Secretária e Médico)
- Definir Disponibilidade (Médico)
- Bloquear Agenda (Secretária e Médico)

3. MÓDULO DE ATENDIMENTO (atendimento-service)

- Agendar Reconsulta (Médico)
- Criar Relatório Clínico (inclui Definir Escopo Médico + Consultar/Editar Relatórios)
- Salvar Dados do Paciente (atualização de prontuário)
- Atribuir a Outro Médico (encaminhamento)
- Definir Urgência (classificação de risco)

Stack Tecnológica

Categoria	Tecnologias
Linguagem	Java 17, Maven
Framework	Spring Boot 3.2, Spring MVC, Spring Data JPA, Spring Security
Banco	MySQL 8.0 (um por microsserviço)
Libs	Lombok, Bean Validation, Jackson, Logbook
Comunicação	OpenFeign (HTTP entre microsserviços)
Documentação	OpenAPI 3.0 / Swagger UI (SpringDoc)

Categoria	Tecnologias
Segurança	JWT (JSON Web Token)
Containers	Docker, Docker Compose
Testes	JUnit 5, Mockito, MockMvc, H2
Versionamento	Git, GitHub, GitFlow
Testes Manuais	Postman

> 📖 **Comece pela Parte 1** → `01_arquitetura_microsservicos.md`

01_arquitetura

🏥 PARTE 1 — VISÃO GERAL DA ARQUITETURA E DEFINIÇÃO DOS MICROSSERVIÇOS

> **Nível:** Intermediário | **Stack:** Java 17 + Spring Boot 3.2 | **Arquitetura:** Microsserviços

---

1. VISÃO GERAL DA ARQUITETURA

1.1 Arquitetura Escolhida

O sistema adota **Arquitetura de Microsserviços** com comunicação **síncrona via REST/HTTP** usando **OpenFeign**. Cada microsserviço é um projeto Spring Boot independente, com banco MySQL isolado (**Database per Service Pattern**).

Os 3 módulos UML mapeiam diretamente para 3 microsserviços:

Módulo UML	Microsserviço	Porta	Banco	Ator(es)
Administrativo	`admin-service`	8081	`admin_db`	Administrador (ADM)
Agendamento	`agendamento-service`	8082	`agendamento_db`	Secretária, Médico
Atendimento	`atendimento-service`	8083	`atendimento_db`	Médico
(suporte)	`commons`	—	—	—

1.2 Por Que Microsserviços?

Aspecto	Monolito	Microsserviços (Escolhido)
Deploy	Tudo junto — um bug trava tudo	Independente por serviço

Aspecto	Monolito	Microsserviços (Escolhido)
Escalabilidade	Escala tudo ou nada	Escala apenas o sobrecarregado
Equipe	Todos no mesmo código	Cada time cuida de um serviço
Resiliência	Falha afeta tudo	Falha isolada por serviço
Complexidade	Simples no início	Maior complexidade infra

Motivação: Empresas como Nubank, iFood e Mercado Livre usam microsserviços. Ao adotar esta arquitetura, os alunos aprendem padrões corporativos reais.

1.3 Diagrama Geral

```
┌──────────────────────────────────────────────────────────────────┐
│                CLIENTE (Browser / Postman)                       │
└──────────────┬──────────────────┬──────────────────┬──────────────┘
               │                  │                  │
               ▼                  ▼                  ▼
      ┌────────────────┐  ┌────────────────────┐  ┌────────────────────┐
      │   admin        │  │  agendamento       │  │  atendimento       │
      │  service       │  │    service         │  │    service         │
      │   :8081        │  │     :8082          │  │     :8083          │
      └──────┬─────────┘  └──────┬─────────────┘  └──────┬─────────────┘
             │     Feign         │      Feign            │
             │◀──────────────────┼◀──────────────────────┼
             ▼                   ▼                       ▼
      ┌─────────────┐   ┌──────────────────┐    ┌──────────────────┐
      │ admin_db    │   │agendamento_db    │    │atendimento_db    │
      │  MySQL      │   │    MySQL         │    │    MySQL         │
      └─────────────┘   └──────────────────┘    └──────────────────┘
```

1.4 Fluxo Geral do Sistema

```
1. ADM cadastra médicos, pacientes, especialidades, convênios → admin-service
2. ADM gerencia perfis de acesso (usuários e permissões) → admin-service
3. Secretária agenda consulta (valida convênio via Feign) → agendamento-service
4. Médico define disponibilidade e consulta agenda → agendamento-service
5. Médico atende paciente, registra relatório clínico → atendimento-service
6. Médico encaminha paciente a outro médico → atendimento-service
```

1.5 Princípios Arquiteturais

Princípio	Aplicação
Single Responsibility	Cada serviço = um domínio UML
Baixo Acoplamento	Comunicação via HTTP, sem banco compartilhado

Princípio	Aplicação
Alta Coesão	Toda lógica de agendamento fica em agendamento-service
Database per Service	Cada serviço tem MySQL isolado
API First	Contratos definidos antes da implementação

---

2. DEFINIÇÃO DOS MICROSSERVIÇOS

2.1 admin-service (Porta 8081) — Módulo Administrativo

Responsabilidades

- CRUD de Médicos
- CRUD de Pacientes
- CRUD de Especialidades
- CRUD de Convênios
- Gerenciamento de Perfis (usuários + permissões)
- Geração de Relatórios Gerenciais

Entidades Principais

- `Medico` (id, nomeCompleto, crm, especialidadeId, ativo, criadoEm, atualizadoEm)
- `Paciente` (id, nomeCompleto, rg, cpf, endereco, cidade, estado, cep, telefone, celular, dataNascimento, genero, possuiConvenio, convenioId, ativo, criadoEm, atualizadoEm)
- `Especialidade` (id, descricao, ativo, criadoEm)
- `Convenio` (id, nomeEmpresa, cnpj, telefone, ativo, criadoEm, atualizadoEm)
- `Usuario` (id, nomeUsuario, senha, perfil, ativo, criadoEm, atualizadoEm)
- `Perfil` (id, nome, descricao) → valores: ADM, MEDICO, SECRETARIA

Endpoints — Médicos

Método	Endpoint	Descrição
`POST`	`/api/v1/medicos`	Cadastrar médico
`GET`	`/api/v1/medicos`	Listar médicos (paginado)
`GET`	`/api/v1/medicos/{id}`	Buscar por ID
`GET`	`/api/v1/medicos/crm/{crm}`	Buscar por CRM
`GET`	`/api/v1/medicos/especialidade/{especialidadeId}`	Listar por especialidade
`PUT`	`/api/v1/medicos/{id}`	Atualizar médico
`DELETE`	`/api/v1/medicos/{id}`	Desativar (soft delete)
`GET`	`/api/v1/medicos/{id}/validar-ativo`	Validar se está ativo (integração)

Endpoints — Pacientes

Método	Endpoint	Descrição
`POST`	`/api/v1/pacientes`	Cadastrar paciente
`GET`	`/api/v1/pacientes`	Listar pacientes (paginado)
`GET`	`/api/v1/pacientes/{id}`	Buscar por ID
`GET`	`/api/v1/pacientes/cpf/{cpf}`	Buscar por CPF
`GET`	`/api/v1/pacientes/buscar?nome={nome}`	Buscar por nome
`PUT`	`/api/v1/pacientes/{id}`	Atualizar
`DELETE`	`/api/v1/pacientes/{id}`	Soft delete
`GET`	`/api/v1/pacientes/{id}/validar-ativo`	Validar se ativo (integração)

Endpoints — Especialidades

Método	Endpoint	Descrição
`POST`	`/api/v1/especialidades`	Criar
`GET`	`/api/v1/especialidades`	Listar todas
`GET`	`/api/v1/especialidades/{id}`	Buscar por ID
`PUT`	`/api/v1/especialidades/{id}`	Atualizar
`DELETE`	`/api/v1/especialidades/{id}`	Soft delete

Endpoints — Convênios

Método	Endpoint	Descrição
`POST`	`/api/v1/convenios`	Criar
`GET`	`/api/v1/convenios`	Listar
`GET`	`/api/v1/convenios/{id}`	Buscar por ID
`PUT`	`/api/v1/convenios/{id}`	Atualizar
`DELETE`	`/api/v1/convenios/{id}`	Soft delete
`GET`	`/api/v1/convenios/{id}/validar-ativo`	Validar (integração)

Endpoints — Perfis e Usuários

Método	Endpoint	Descrição
`POST`	`/api/v1/auth/login`	Autenticar usuário
`POST`	`/api/v1/usuarios`	Criar usuário
`GET`	`/api/v1/usuarios`	Listar usuários

Método	Endpoint	Descrição
`PUT`	`/api/v1/usuarios/{id}`	Atualizar
`PUT`	`/api/v1/usuarios/{id}/perfil`	Atualizar permissões
`DELETE`	`/api/v1/usuarios/{id}`	Desativar

Endpoints — Relatórios Gerenciais

Método	Endpoint	Descrição
`GET`	`/api/v1/relatorios/consultas-por-periodo?inicio={}&fim={}`	Consultas por período
`GET`	`/api/v1/relatorios/consultas-por-medico?medicoId={}&inicio={}&fim={}`	Consultas por médico
`GET`	`/api/v1/relatorios/consultas-por-especialidade`	Por especialidade
`GET`	`/api/v1/relatorios/convenios-utilizados`	Convênios mais utilizados
`GET`	`/api/v1/relatorios/pacientes-cadastrados?inicio={}&fim={}`	Pacientes novos

Regras de Negócio

1. CPF e RG devem ser únicos para pacientes
2. CRM deve ser único para médicos
3. CNPJ deve ser único para convênios
4. CPF e CNPJ devem ser validados (algoritmo)
5. Exclusões são lógicas (soft delete — campo `ativo = false`)
6. Médico deve ter especialidade válida cadastrada
7. Senha armazenada com BCrypt (strength 12)
8. Relatórios consultam dados via Feign nos demais serviços

Exemplo de Payload — Cadastrar Médico

Request:

```
{
  "nomeCompleto": "Dra. Maria Souza",
  "crm": "CRM/SP 12345",
  "especialidadeId": 1
}
```

Response (201):

```
{
  "id": 3,
```

```
  "nomeCompleto": "Dra. Maria Souza",
  "crm": "CRM/SP 12345",
  "especialidade": "Cardiologia",
  "ativo": true,
  "criadoEm": "2026-05-09T10:30:00"
}
```

Dependências (Feign)

- `agendamento-service` → para relatórios de consultas por período
- `atendimento-service` → para relatórios clínicos

2.2 agendamento-service (Porta 8082) — Módulo de Agendamento

Responsabilidades

- Agendar Consulta (Secretária) com validação de convênio
- Remarcar Consulta (Secretária)
- Cancelar Consulta (Secretária)
- Consultar Agenda (Secretária e Médico)
- Definir Disponibilidade (Médico)
- Bloquear Agenda (Secretária e Médico)

Entidades

- `Consulta` (id, pacienteId, medicoId, dataHora, status, tipo, consultaOriginalId, criadoEm, atualizadoEm)
- `Cancelamento` (id, consultaId, motivo, canceladoPor, criadoEm)
- `DisponibilidadeMedico` (id, medicoId, diaSemana, horaInicio, horaFim, ativo)
- `BloqueioAgenda` (id, medicoId, dataInicio, dataFim, motivo, bloqueadoPor, criadoEm)

Status da Consulta

```
AGENDADA → CONFIRMADA → EM_ATENDIMENTO → CONCLUIDA
                ↓
           CANCELADA
                ↓
           REMARCADA
```

Endpoints — Consultas

Método	Endpoint	Descrição
`POST`	`/api/v1/consultas`	Agendar consulta
`GET`	`/api/v1/consultas`	Listar consultas (filtros)
`GET`	`/api/v1/consultas/{id}`	Buscar por ID
`GET`	`/api/v1/consultas/medico/{medicoId}`	Listar por médico
`GET`	`/api/v1/consultas/paciente/{pacienteId}`	Listar por paciente
`PUT`	`/api/v1/consultas/{id}/remarcar`	Remarcar consulta
`PUT`	`/api/v1/consultas/{id}/cancelar`	Cancelar consulta
`PUT`	`/api/v1/consultas/{id}/status`	Atualizar status
`GET`	`/api/v1/consultas/horarios-disponiveis`	Horários livres

Endpoints — Disponibilidade

Método	Endpoint	Descrição
`POST`	`/api/v1/disponibilidades`	Definir horário de trabalho
`GET`	`/api/v1/disponibilidades/medico/{medicoId}`	Consultar disponibilidade
`PUT`	`/api/v1/disponibilidades/{id}`	Atualizar
`DELETE`	`/api/v1/disponibilidades/{id}`	Remover

Endpoints — Bloqueio de Agenda

Método	Endpoint	Descrição
`POST`	`/api/v1/bloqueios`	Bloquear período
`GET`	`/api/v1/bloqueios/medico/{medicoId}`	Listar bloqueios
`DELETE`	`/api/v1/bloqueios/{id}`	Remover bloqueio

Regras de Negócio

1. Paciente deve estar ativo (valida via Feign → admin-service)
2. Médico deve estar ativo (valida via Feign → admin-service)
3. Se paciente possui convênio, **obrigatório validar convênio** via Feign
4. Não pode haver conflito de horário para o mesmo médico
5. Não pode agendar em horário bloqueado
6. Não pode agendar fora da disponibilidade do médico
7. Consultas duram 30 minutos por padrão
8. Cancelamento exige motivo obrigatório
9. Cancelamento libera o horário automaticamente
10. Remarcar = cancelar atual + criar nova consulta vinculada
11. Bloqueio impede novos agendamentos no período

Exemplo de Payload — Agendar Consulta

Request:

```
{
  "pacienteId": 1,
  "medicoId": 3,
  "dataHora": "2026-06-15T10:00:00",
  "tipo": "PRIMEIRA_CONSULTA"
}
```

Response (201):

```
{
  "id": 42,
  "pacienteId": 1,
  "nomePaciente": "João da Silva",
  "medicoId": 3,
  "nomeMedico": "Dra. Maria Souza",
  "crmMedico": "CRM/SP 12345",
  "especialidade": "Cardiologia",
  "dataHora": "2026-06-15T10:00:00",
  "status": "AGENDADA",
  "tipo": "PRIMEIRA_CONSULTA",
  "criadoEm": "2026-05-09T09:30:00"
}
```

Dependências (Feign)

- `admin-service` → validar paciente ativo, médico ativo, convênio ativo

2.3 atendimento-service (Porta 8083) — Módulo de Atendimento

Responsabilidades

- Agendar Reconsulta (retorno)
- Criar Relatório Clínico (com escopo médico + consultar/editar anteriores)
- Salvar Dados do Paciente (atualizar prontuário)
- Atribuir a Outro Médico (encaminhamento)
- Definir Urgência (classificação de risco)

Entidades

- `Prontuario` (id, pacienteId, criadoEm)
- `Atendimento` (id, prontuarioId, consultaId, medicoId, dataAtendimento, sintomas, diagnostico, escopoMedico, observacoes, nivelUrgencia, criadoEm, atualizadoEm)
- `RelatorioClinico` (id, atendimentoId, conteudo, criadoEm, atualizadoEm)
- `Encaminhamento` (id, atendimentoId, medicoOrigemId, medicoDestinoId, motivo, especialidadeDestino, prioridade, status, criadoEm)
- `Receita` (id, atendimentoId, medicamento, dosagem, frequencia, duracao, observacoes, criadoEm)
- `SolicitacaoExame` (id, atendimentoId, tipoExame, descricao, urgencia, status, criadoEm)

Níveis de Urgência (Enum)

```
VERDE (Não urgente) → AMARELO (Pouco urgente) → LARANJA (Urgente) → VERMELHO
(Emergência)
```

Status do Encaminhamento

```
PENDENTE → ACEITO → EM_ATENDIMENTO → CONCLUIDO
```

Endpoints — Atendimento

Método	Endpoint	Descrição
`POST`	`/api/v1/atendimentos`	Registrar atendimento
`GET`	`/api/v1/atendimentos/{id}`	Buscar atendimento
`GET`	`/api/v1/atendimentos/paciente/{pacienteId}`	Histórico do paciente
`PUT`	`/api/v1/atendimentos/{id}`	Atualizar dados (salvar prontuário)
`PUT`	`/api/v1/atendimentos/{id}/urgencia`	Definir urgência

Endpoints — Relatório Clínico

Método	Endpoint	Descrição
`POST`	`/api/v1/relatorios-clinicos`	Criar relatório
`GET`	`/api/v1/relatorios-clinicos/{id}`	Consultar relatório
`GET`	`/api/v1/relatorios-clinicos/paciente/{pacienteId}`	Relatórios do paciente
`PUT`	`/api/v1/relatorios-clinicos/{id}`	Editar relatório

Endpoints — Encaminhamento

Método	Endpoint	Descrição
`POST`	`/api/v1/encaminhamentos`	Atribuir a outro médico

Método	Endpoint	Descrição
`GET`	`/api/v1/encaminhamentos/{id}`	Buscar encaminhamento
`GET`	`/api/v1/encaminhamentos/medico-destino/{medicoId}`	Encaminhamentos recebidos
`PUT`	`/api/v1/encaminhamentos/{id}/status`	Atualizar status

Endpoints — Reconsulta

Método	Endpoint	Descrição
`POST`	`/api/v1/reconsultas`	Agendar reconsulta/retorno

Regras de Negócio

1. Prontuário é criado automaticamente no primeiro atendimento
2. Atendimento é vinculado a uma consulta com status `EM_ATENDIMENTO`
3. Apenas médicos podem registrar atendimentos
4. Relatório clínico inclui definição de escopo médico obrigatória
5. Relatórios anteriores podem ser consultados e editados
6. Encaminhamento exige: médico destino, motivo, especialidade, prioridade
7. Médico destino é validado via Feign → admin-service
8. Reconsulta cria novo agendamento via Feign → agendamento-service com tipo RETORNO
9. Urgência usa escala de cores: VERDE, AMARELO, LARANJA, VERMELHO
10. Salvar dados do paciente atualiza o prontuário base

Exemplo de Payload — Registrar Atendimento

Request:

```json
{
  "consultaId": 42,
  "medicoId": 3,
  "pacienteId": 1,
  "sintomas": "Dor no peito ao esforço, falta de ar",
  "diagnostico": "Angina estável — CID I20.8",
  "escopoMedico": "Avaliação cardiológica completa",
  "observacoes": "Paciente relata início dos sintomas há 2 semanas",
  "nivelUrgencia": "LARANJA",
  "receitas": [
    {
      "medicamento": "Isordil 5mg",
      "dosagem": "1 comprimido",
      "frequencia": "Sublingual em caso de dor",
      "duracao": "Uso contínuo"
    }
```

```
  ],
  "solicitacoesExame": [
    {
      "tipoExame": "ECG",
      "descricao": "Eletrocardiograma de repouso",
      "urgencia": "URGENTE"
    }
  ]
}
```

Exemplo — Atribuir a Outro Médico

Request:

```
{
  "atendimentoId": 10,
  "medicoOrigemId": 3,
  "medicoDestinoId": 7,
  "motivo": "Paciente necessita avaliação neurológica especializada",
  "especialidadeDestino": "Neurologia",
  "prioridade": "ALTA"
}
```

Dependências (Feign)

- `admin-service` → validar médico destino, buscar dados paciente/médico
- `agendamento-service` → validar consulta, criar reconsulta

2.4 commons (Módulo Maven Compartilhado)

Responsabilidades

- DTOs compartilhados (RespostaErro, RespostaPaginada)
- Exception handlers globais
- Utilitários (ValidadorCpf, ValidadorCnpj)
- Configurações comuns (Logbook, Jackson)
- Constantes da API

Estrutura

```
commons/
├── src/main/java/br/com/clinica/commons/
│   ├── dto/
│   │   ├── RespostaErro.java
│   │   ├── RespostaPaginada.java
```

```
│   │   └── ErroValidacao.java
│   ├── exception/
│   │   ├── RegraDeNegocioException.java
│   │   ├── RecursoNaoEncontradoException.java
│   │   ├── RecursoDuplicadoException.java
│   │   ├── IntegracaoException.java
│   │   └── TratadorGlobalExcecoes.java
│   ├── config/
│   │   ├── LogbookConfig.java
│   │   └── JacksonConfig.java
│   ├── util/
│   │   ├── ValidadorCpf.java
│   │   ├── ValidadorCnpj.java
│   │   └── DataUtils.java
│   └── constantes/
│       └── ConstantesApi.java
└── pom.xml
```

2.5 Matriz de Comunicação entre Serviços

Origem	Destino	Endpoint Consumido	Motivo
`agendamento-service`	`admin-service`	`GET /api/v1/pacientes/{id}/validar-ativo`	Validar paciente ao agendar
`agendamento-service`	`admin-service`	`GET /api/v1/medicos/{id}/validar-ativo`	Validar médico ao agendar
`agendamento-service`	`admin-service`	`GET /api/v1/convenios/{id}/validar-ativo`	Validar convênio ao agendar
`atendimento-service`	`admin-service`	`GET /api/v1/pacientes/{id}`	Buscar dados do paciente
`atendimento-service`	`admin-service`	`GET /api/v1/medicos/{id}`	Validar médico destino
`atendimento-service`	`agendamento-service`	`GET /api/v1/consultas/{id}`	Validar consulta
`atendimento-service`	`agendamento-service`	`POST /api/v1/consultas`	Criar reconsulta
`admin-service`	`agendamento-service`	`GET /api/v1/consultas?periodo=...`	Relatórios gerenciais

Continua na Parte 2 → Modelagem de Banco de Dados e Estrutura de Pastas

02_banco_estrutura

🗂️ PARTE 2 — MODELAGEM DO BANCO DE DADOS E ESTRUTURA DE PASTAS

---

3. MODELAGEM DO BANCO DE DADOS

3.1 Estratégia: Database per Service

Cada microsserviço possui banco **isolado**. Nenhum serviço acessa diretamente o banco de outro.

Serviço	Banco	Porta Host
`admin-service`	`admin_db`	3306
`agendamento-service`	`agendamento_db`	3307
`atendimento-service`	`atendimento_db`	3308

> **Por quê?** Bancos isolados garantem que uma migration mal feita em um serviço não derrube os outros.

3.2 Diagrama ER — admin_db

```
┌──────────────────────────────┐
│         usuarios             │
├──────────────────────────────┤
│ id             BIGINT PK     │     ┌──────────────────────────┐
│ nome_usuario   VARCHAR(100)  │     │        perfis            │
│ senha          VARCHAR(255)  │     ├──────────────────────────┤
│ perfil_id      BIGINT FK─────┼────▶│ id         BIGINT PK     │
│ ativo          BOOLEAN       │     │ nome       VARCHAR(50)   │
│ criado_em      TIMESTAMP     │     │ descricao  VARCHAR(200   │
│ atualizado_em  TIMESTAMP     │     │ criado_em  TIMESTAMP     │
└──────────────────────────────┘     └──────────────────────────┘


┌──────────────────────────────┐
│       especialidades         │
├──────────────────────────────┤
│ id           BIGINT PK       │
│ descricao    VARCHAR(100)    │
│ ativo        BOOLEAN         │
│ criado_em    TIMESTAMP       │
└──────────────────────────────┘
```

```
┌──────────────────────────────────┐     ┌──────────────────────────┐
│         medicos                  │     │   especialidades         │
├──────────────────────────────────┤     │                          │
│ id              BIGINT PK        │     │                          │
│ nome_completo   VARCHAR(150)     │     │                          │
│ crm             VARCHAR(20)      │     │                          │
│ especialidade_id BIGINT FK ──────┼────►│                          │
│ ativo           BOOLEAN          │     └──────────────────────────┘
│ criado_em       TIMESTAMP        │
│ atualizado_em   TIMESTAMP        │
└──────────────────────────────────┘


┌──────────────────────────────────┐
│        pacientes                 │
├──────────────────────────────────┤
│ id              BIGINT PK        │
│ nome_completo   VARCHAR(150)     │
│ rg              VARCHAR(20)      │
│ cpf             VARCHAR(14)      │
│ endereco        VARCHAR(255)     │
│ bairro          VARCHAR(100)     │
│ cidade          VARCHAR(100)     │
│ estado          VARCHAR(2)       │
│ cep             VARCHAR(10)      │
│ telefone        VARCHAR(20)      │
│ celular         VARCHAR(20)      │
│ data_nascimento DATE             │
│ genero          ENUM             │
│ possui_convenio BOOLEAN          │
│ convenio_id     BIGINT           │
│ ativo           BOOLEAN          │
│ criado_em       TIMESTAMP        │
│ atualizado_em   TIMESTAMP        │
└──────────────────────────────────┘


┌──────────────────────────────────┐
│        convenios                 │
├──────────────────────────────────┤
│ id              BIGINT PK        │
│ nome_empresa    VARCHAR(150)     │
│ cnpj            VARCHAR(18)      │
│ telefone        VARCHAR(20)      │
│ ativo           BOOLEAN          │
│ criado_em       TIMESTAMP        │
│ atualizado_em   TIMESTAMP        │
└──────────────────────────────────┘
```

SQL — admin_db

```sql
CREATE DATABASE IF NOT EXISTS admin_db;
USE admin_db;

CREATE TABLE perfis (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(50) NOT NULL UNIQUE,
    descricao VARCHAR(200),
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE usuarios (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    nome_usuario VARCHAR(100) NOT NULL UNIQUE,
    senha VARCHAR(255) NOT NULL,
    perfil_id BIGINT NOT NULL,
    ativo BOOLEAN DEFAULT TRUE,
    tentativas_login INT DEFAULT 0,
    bloqueado_ate TIMESTAMP NULL,
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    atualizado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_usuario_perfil FOREIGN KEY (perfil_id) REFERENCES perfis(id),
    INDEX idx_nome_usuario (nome_usuario)
);

CREATE TABLE especialidades (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    descricao VARCHAR(100) NOT NULL UNIQUE,
    ativo BOOLEAN DEFAULT TRUE,
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE medicos (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    nome_completo VARCHAR(150) NOT NULL,
    crm VARCHAR(20) NOT NULL UNIQUE,
    especialidade_id BIGINT NOT NULL,
    ativo BOOLEAN DEFAULT TRUE,
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    atualizado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_medico_especialidade FOREIGN KEY (especialidade_id) REFERENCES
especialidades(id),
    INDEX idx_crm (crm),
    INDEX idx_especialidade (especialidade_id)
);
```

```sql
CREATE TABLE convenios (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    nome_empresa VARCHAR(150) NOT NULL,
    cnpj VARCHAR(18) NOT NULL UNIQUE,
    telefone VARCHAR(20),
    ativo BOOLEAN DEFAULT TRUE,
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    atualizado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    INDEX idx_cnpj (cnpj)
);

CREATE TABLE pacientes (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    nome_completo VARCHAR(150) NOT NULL,
    rg VARCHAR(20) NOT NULL UNIQUE,
    cpf VARCHAR(14) NOT NULL UNIQUE,
    endereco VARCHAR(255) NOT NULL,
    bairro VARCHAR(100) NOT NULL,
    cidade VARCHAR(100) NOT NULL,
    estado VARCHAR(2) NOT NULL,
    cep VARCHAR(10) NOT NULL,
    telefone VARCHAR(20),
    celular VARCHAR(20) NOT NULL,
    data_nascimento DATE NOT NULL,
    genero ENUM('MASCULINO', 'FEMININO', 'OUTRO') NOT NULL,
    possui_convenio BOOLEAN DEFAULT FALSE,
    convenio_id BIGINT NULL,
    ativo BOOLEAN DEFAULT TRUE,
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    atualizado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_paciente_convenio FOREIGN KEY (convenio_id) REFERENCES
convenios(id),
    INDEX idx_cpf (cpf),
    INDEX idx_rg (rg),
    INDEX idx_nome (nome_completo)
);

-- Dados iniciais
INSERT INTO perfis (nome, descricao) VALUES
('ADM', 'Administrador do sistema'),
('MEDICO', 'Médico'),
('SECRETARIA', 'Secretária/Recepcionista');

INSERT INTO especialidades (descricao) VALUES
('Cardiologia'), ('Dermatologia'), ('Ortopedia'),
('Pediatria'), ('Ginecologia'), ('Neurologia'),
```

```
('Oftalmologia'), ('Urologia'), ('Psiquiatria'),
('Clínico Geral');
```

3.3 Diagrama ER — agendamento_db

```
┌──────────────────────────────────┐      ┌──────────────────────────────────┐
│          consultas               │      │      cancelamentos               │
├──────────────────────────────────┤      ├──────────────────────────────────┤
│ id              BIGINT PK        │◄─────│ consulta_id  BIGINT FK           │
│ paciente_id     BIGINT           │      │ id           BIGINT PK           │
│ medico_id       BIGINT           │      │ motivo       TEXT                │
│ data_hora       DATETIME         │      │ cancelado_por VARCHAR(100)       │
│ status          ENUM             │      │ criado_em    TIMESTAMP           │
│ tipo            ENUM             │      └──────────────────────────────────┘
│ consulta_original_id BIGINT FK(self)│
│ criado_em       TIMESTAMP        │
│ atualizado_em   TIMESTAMP        │
└──────────────────────────────────┘


┌──────────────────────────────────┐      ┌──────────────────────────────────┐
│ disponibilidades_medico          │      │   bloqueios_agenda               │
├──────────────────────────────────┤      ├──────────────────────────────────┤
│ id           BIGINT PK           │      │ id           BIGINT PK           │
│ medico_id    BIGINT              │      │ medico_id    BIGINT              │
│ dia_semana   ENUM                │      │ data_inicio  DATETIME            │
│ hora_inicio  TIME                │      │ data_fim     DATETIME            │
│ hora_fim     TIME                │      │ motivo       VARCHAR(255)        │
│ ativo        BOOLEAN             │      │ bloqueado_por VARCHAR(100)       │
│ criado_em    TIMESTAMP           │      │ criado_em    TIMESTAMP           │
└──────────────────────────────────┘      └──────────────────────────────────┘
```

SQL — agendamento_db

```
CREATE DATABASE IF NOT EXISTS agendamento_db;
USE agendamento_db;

CREATE TABLE consultas (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    paciente_id BIGINT NOT NULL,
    medico_id BIGINT NOT NULL,
    data_hora DATETIME NOT NULL,
    status
ENUM('AGENDADA','CONFIRMADA','EM_ATENDIMENTO','CONCLUIDA','CANCELADA','REMARCADA')
DEFAULT 'AGENDADA',
    tipo ENUM('PRIMEIRA_CONSULTA','ACOMPANHAMENTO','RETORNO') DEFAULT
'PRIMEIRA_CONSULTA',
```

```
    consulta_original_id BIGINT NULL,
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    atualizado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_consulta_original FOREIGN KEY (consulta_original_id) REFERENCES
consultas(id),
    INDEX idx_paciente (paciente_id),
    INDEX idx_medico (medico_id),
    INDEX idx_data_hora (data_hora),
    INDEX idx_status (status),
    UNIQUE INDEX idx_medico_horario (medico_id, data_hora)
);

CREATE TABLE cancelamentos (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    consulta_id BIGINT NOT NULL UNIQUE,
    motivo TEXT NOT NULL,
    cancelado_por VARCHAR(100) NOT NULL,
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_cancelamento_consulta FOREIGN KEY (consulta_id) REFERENCES
consultas(id)
);

CREATE TABLE disponibilidades_medico (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    medico_id BIGINT NOT NULL,
    dia_semana ENUM('SEGUNDA','TERCA','QUARTA','QUINTA','SEXTA','SABADO') NOT
NULL,
    hora_inicio TIME NOT NULL,
    hora_fim TIME NOT NULL,
    ativo BOOLEAN DEFAULT TRUE,
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_medico_dia (medico_id, dia_semana)
);

CREATE TABLE bloqueios_agenda (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    medico_id BIGINT NOT NULL,
    data_inicio DATETIME NOT NULL,
    data_fim DATETIME NOT NULL,
    motivo VARCHAR(255) NOT NULL,
    bloqueado_por VARCHAR(100) NOT NULL,
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_medico_periodo (medico_id, data_inicio, data_fim)
);
```

3.4 Diagrama ER — atendimento_db

```
┌──────────────────────────────┐
│      prontuarios             │
├──────────────────────────────┤      ┌──────────────────────────────┐
│ id          BIGINT PK        │◀─────│      atendimentos            │
│ paciente_id BIGINT UQ        │      ├──────────────────────────────┤
│ criado_em   TIMESTAMP        │      │ id            BIGINT PK      │
└──────────────────────────────┘      │ prontuario_id BIGINT FK      │
                                      │ consulta_id   BIGINT         │
        ┌──────────────────────────────│ medico_id     BIGINT         │
        │                             │ data          DATETIME       │
        │                             │ sintomas      TEXT           │
        │                             │ diagnostico   TEXT           │
        │                             │ escopo_medico TEXT           │
        │                             │ observacoes   TEXT           │
        │                             │ nivel_urgencia ENUM          │
        │                             │ criado_em     TIMESTAMP      │
        │                             │ atualizado_em TIMESTAMP      │
        │                             └──────────────────────────────┘
        ▼

┌──────────────────────────┐   ┌──────────────────────────┐
┌──────────────────────────┐
│ relatorios_clinicos      │   │      receitas            │   │ solicitacoes_exame       │
├──────────────────────────┤   ├──────────────────────────┤
├──────────────────────────┤
│ id        BIGINT PK      │   │ id        BIGINT PK      │   │ id        BIGINT PK      │
│ atend_id  BIGINT FK      │   │ atend_id  BIGINT FK      │   │ atend_id  BIGINT FK      │
│ conteudo  TEXT           │   │ medicamento VARCHAR      │   │ tipo_exame VARCHAR       │
│ criado_em TIMESTAMP      │   │ dosagem   VARCHAR        │   │ descricao TEXT           │
│ atualizado_em TS         │   │ frequencia VARCHAR       │   │ urgencia ENUM            │
└──────────────────────────┘   │ duracao   VARCHAR        │   │ status    ENUM           │
                               │ observacoes TEXT         │   │ criado_em TIMESTAMP      │
┌──────────────────────────┐   │ criado_em TIMESTAMP      │
└──────────────────────────┘

│   encaminhamentos        │   └──────────────────────────┘
├──────────────────────────┤
│ id            BIGINT PK  │
│ atend_id      BIGINT FK  │
│ medico_origem BIGINT     │
│ medico_destino BIGINT    │
│ motivo        TEXT       │
│ especialidade VARCHAR    │
│ prioridade    ENUM       │
│ status        ENUM       │
│ criado_em     TS         │
└──────────────────────────┘
```

SQL — atendimento_db

```sql
CREATE DATABASE IF NOT EXISTS atendimento_db;
USE atendimento_db;

CREATE TABLE prontuarios (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    paciente_id BIGINT NOT NULL UNIQUE,
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_paciente (paciente_id)
);

CREATE TABLE atendimentos (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    prontuario_id BIGINT NOT NULL,
    consulta_id BIGINT NOT NULL,
    medico_id BIGINT NOT NULL,
    data_atendimento DATETIME NOT NULL,
    sintomas TEXT,
    diagnostico TEXT,
    escopo_medico TEXT NOT NULL,
    observacoes TEXT,
    nivel_urgencia ENUM('VERDE','AMARELO','LARANJA','VERMELHO') DEFAULT 'VERDE',
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    atualizado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_atendimento_prontuario FOREIGN KEY (prontuario_id) REFERENCES
prontuarios(id),
    INDEX idx_prontuario (prontuario_id),
    INDEX idx_consulta (consulta_id),
    INDEX idx_medico (medico_id)
);

CREATE TABLE relatorios_clinicos (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    atendimento_id BIGINT NOT NULL,
    conteudo TEXT NOT NULL,
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    atualizado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_relatorio_atendimento FOREIGN KEY (atendimento_id) REFERENCES
atendimentos(id)
);

CREATE TABLE receitas (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    atendimento_id BIGINT NOT NULL,
    medicamento VARCHAR(200) NOT NULL,
    dosagem VARCHAR(100) NOT NULL,
    frequencia VARCHAR(100) NOT NULL,
    duracao VARCHAR(100) NOT NULL,
```

```
    observacoes TEXT,
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_receita_atendimento FOREIGN KEY (atendimento_id) REFERENCES
atendimentos(id)
);

CREATE TABLE solicitacoes_exame (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    atendimento_id BIGINT NOT NULL,
    tipo_exame VARCHAR(100) NOT NULL,
    descricao TEXT NOT NULL,
    urgencia ENUM('NORMAL','URGENTE','EMERGENCIAL') DEFAULT 'NORMAL',
    status ENUM('SOLICITADO','AGENDADO','CONCLUIDO') DEFAULT 'SOLICITADO',
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_exame_atendimento FOREIGN KEY (atendimento_id) REFERENCES
atendimentos(id)
);

CREATE TABLE encaminhamentos (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    atendimento_id BIGINT NOT NULL,
    medico_origem_id BIGINT NOT NULL,
    medico_destino_id BIGINT NOT NULL,
    motivo TEXT NOT NULL,
    especialidade_destino VARCHAR(100) NOT NULL,
    prioridade ENUM('BAIXA','MEDIA','ALTA','URGENTE') DEFAULT 'MEDIA',
    status ENUM('PENDENTE','ACEITO','EM_ATENDIMENTO','CONCLUIDO') DEFAULT
'PENDENTE',
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_encaminhamento_atendimento FOREIGN KEY (atendimento_id)
REFERENCES atendimentos(id),
    INDEX idx_medico_destino (medico_destino_id),
    INDEX idx_status (status)
);
```

---

4. ESTRUTURA DE PASTAS

4.1 Estrutura Raiz

```
clinica-medica/
├── commons/
├── admin-service/
├── agendamento-service/
├── atendimento-service/
├── docker-compose.yml
```

```
├── build-and-run.sh
├── .gitignore
├── README.md
└── docs/
    ├── arquitetura.md
    ├── contratos-api.md
    ├── modelo-banco.md
    └── postman/
        ├── admin-service.postman_collection.json
        ├── agendamento-service.postman_collection.json
        └── atendimento-service.postman_collection.json
```

4.2 Estrutura Padrão de Microsserviço (exemplo: admin-service)

```
admin-service/
├── src/
│   ├── main/
│   │   ├── java/br/com/clinica/admin/
│   │   │   ├── AdminServiceApplication.java
│   │   │   ├── controller/
│   │   │   │   ├── MedicoController.java
│   │   │   │   ├── PacienteController.java
│   │   │   │   ├── EspecialidadeController.java
│   │   │   │   ├── ConvenioController.java
│   │   │   │   ├── UsuarioController.java
│   │   │   │   └── RelatorioController.java
│   │   │   ├── service/
│   │   │   │   ├── MedicoService.java
│   │   │   │   ├── PacienteService.java
│   │   │   │   ├── EspecialidadeService.java
│   │   │   │   ├── ConvenioService.java
│   │   │   │   ├── UsuarioService.java
│   │   │   │   └── RelatorioService.java
│   │   │   ├── repository/
│   │   │   │   ├── MedicoRepository.java
│   │   │   │   ├── PacienteRepository.java
│   │   │   │   ├── EspecialidadeRepository.java
│   │   │   │   ├── ConvenioRepository.java
│   │   │   │   └── UsuarioRepository.java
│   │   │   ├── entity/
│   │   │   │   ├── Medico.java
│   │   │   │   ├── Paciente.java
│   │   │   │   ├── Especialidade.java
│   │   │   │   ├── Convenio.java
│   │   │   │   ├── Usuario.java
```

```
│   │   │   │   └── Perfil.java
│   │   │   ├── dto/
│   │   │   │   ├── requisicao/
│   │   │   │   │   ├── MedicoRequisicao.java
│   │   │   │   │   ├── PacienteRequisicao.java
│   │   │   │   │   ├── EspecialidadeRequisicao.java
│   │   │   │   │   └── ConvenioRequisicao.java
│   │   │   │   └── resposta/
│   │   │   │       ├── MedicoResposta.java
│   │   │   │       ├── PacienteResposta.java
│   │   │   │       ├── EspecialidadeResposta.java
│   │   │   │       └── ConvenioResposta.java
│   │   │   ├── mapper/
│   │   │   │   ├── MedicoMapper.java
│   │   │   │   ├── PacienteMapper.java
│   │   │   │   ├── EspecialidadeMapper.java
│   │   │   │   └── ConvenioMapper.java
│   │   │   ├── config/
│   │   │   │   ├── SwaggerConfig.java
│   │   │   │   ├── SecurityConfig.java
│   │   │   │   └── WebConfig.java
│   │   │   ├── security/
│   │   │   │   ├── JwtService.java
│   │   │   │   └── JwtFiltroAutenticacao.java
│   │   │   └── validacao/
│   │   │       ├── CpfValido.java
│   │   │       └── ValidadorCpf.java
│   │   └── resources/
│   │       ├── application.yml
│   │       ├── application-dev.yml
│   │       └── application-prod.yml
│   └── test/
│       └── java/br/com/clinica/admin/
│           ├── controller/
│           │   └── MedicoControllerTest.java
│           └── service/
│               └── MedicoServiceTest.java
├── Dockerfile
└── pom.xml
```

4.3 Papel de Cada Camada

```
Request HTTP → Controller → Service → Repository → Database
```

Camada	Responsabilidade	Regra
Controller	Recebe HTTP, valida DTO, retorna response	NUNCA contém lógica de negócio
Service	Regras de negócio, validações, orquestração	NUNCA acessa HttpServletRequest
Repository	Interface JPA para acesso a dados	NUNCA contém lógica de negócio
Entity	Mapeamento JPA da tabela	NUNCA exposta na API
DTO Requisicao	Dados recebidos do cliente	Contém validações Bean Validation
DTO Resposta	Dados retornados ao cliente	Apenas campos necessários
Mapper	Converte Entity ↔ DTO	Classe estática ou @Component

4.4 application.yml do admin-service

```yaml
server:
  port: 8081

spring:
  application:
    name: admin-service
  datasource:
    url: jdbc:mysql://${DB_HOST:localhost}:${DB_PORT:3306}/admin_db
    username: ${DB_USERNAME:root}
    password: ${DB_PASSWORD:root}
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        format_sql: true
        dialect: org.hibernate.dialect.MySQLDialect

springdoc:
  api-docs:
    path: /api-docs
  swagger-ui:
    path: /swagger-ui.html

logging:
  level:
    br.com.clinica: DEBUG
```

```
  pattern:
    console: "%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36} - %msg%n"
```

Continua na Parte 3 → Padronização Corporativa, Cronograma e Tarefas

03_padronizacao_tarefas

📋 PARTE 3 — PADRONIZAÇÃO, CRONOGRAMA, EQUIPE E TAREFAS

---

5. PADRONIZAÇÃO CORPORATIVA

5.1 Convenção de Nomes — Classes Java

Tipo	Convenção	Exemplo
Entity	`NomeSingularPT`	`Medico`, `Paciente`, `Consulta`
Repository	`NomeRepository`	`MedicoRepository`
Service	`NomeService`	`MedicoService`
Controller	`NomeController`	`MedicoController`
DTO Request	`NomeRequisicao`	`MedicoRequisicao`
DTO Response	`NomeResposta`	`MedicoResposta`
Mapper	`NomeMapper`	`MedicoMapper`
Feign Client	`NomeServiceClient`	`AdminServiceClient`
Enum	`NomeMaiusculo`	`StatusConsulta`, `NivelUrgencia`

5.2 Pacotes Java

```
br.com.clinica.<servico>.<camada>
```

Exemplos: `br.com.clinica.admin.controller`, `br.com.clinica.agendamento.service`

5.3 Variáveis e Métodos

Tipo	Convenção	Exemplo
Variável local	camelCase	`nomePaciente`, `dataConsulta`
Constante	UPPER_SNAKE_CASE	`MAXIMO_TENTATIVAS_LOGIN`
Método	camelCase, verbo	`buscarPorId()`, `criarMedico()`

Tipo	Convenção	Exemplo
Boolean	prefixo is/possui	`isAtivo`, `possuiConvenio`

5.4 Endpoints REST

Padrão: `/api/v1/<recurso-no-plural-pt>`

Regras:

- Sempre plural: `/pacientes`, não `/paciente`
- Sempre lowercase com hífens: `/relatorios-clinicos`
- Versão na URL: `/api/v1/`
- Nunca verbos: ~~`/api/v1/buscarPacientes`~~

5.5 Padrão de Commits (Conventional Commits)

```
<tipo>(<escopo>): <mensagem curta>
```

Tipo	Uso	Exemplo
`feat`	Nova funcionalidade	`feat(admin): criar CRUD de médicos`
`fix`	Correção de bug	`fix(agendamento): corrigir validação de conflito`
`docs`	Documentação	`docs(admin): adicionar anotações Swagger`
`test`	Testes	`test(admin): adicionar testes unitários MedicoService`
`chore`	Build, configs	`chore(docker): adicionar docker-compose`
`refactor`	Refatoração	`refactor(commons): extrair validador de CPF`

5.6 Branches (GitFlow)

```
main              ← Produção (tags de release)
  └── develop     ← Integração
      ├── feature/admin-crud-medicos
      ├── feature/agendamento-consultas
      ├── feature/atendimento-prontuario
      ├── bugfix/corrigir-validacao-cpf
      └── hotfix/corrigir-falha-login
```

Regras: Nunca commitar direto em `main` ou `develop`. PR obrigatório. Mínimo 1 revisão.

5.7 Versionamento Semântico

`MAJOR.MINOR.PATCH` → Versão inicial do projeto: **1.0.0**

6. ORDEM IDEAL DE DESENVOLVIMENTO

```
FASE 1: Fundação
    └── commons → base compartilhada
    └── Docker Compose → bancos MySQL

FASE 2: Domínio Base (independente)
    └── admin-service → não depende de ninguém

FASE 3: Domínio Dependente
    └── agendamento-service → depende de admin-service

FASE 4: Domínio Complexo
    └── atendimento-service → depende de admin + agendamento
```

Fase	Serviço	Justificativa
1	`commons`	Exceptions e DTOs compartilhados — precisa existir primeiro
1	Docker Compose	Bancos precisam estar rodando
2	`admin-service`	Não depende de ninguém. Fornece médicos, pacientes, convênios
3	`agendamento-service`	Depende de admin para validar paciente/médico/convênio
4	`atendimento-service`	Depende de admin e agendamento — é o mais complexo

---

7. CRONOGRAMA (8 Semanas)

Semana	Sprint	Entregas
1	Sprint 0 — Setup	Infra, Docker, commons, estrutura dos projetos
2	Sprint 1 — Admin Base	CRUD médicos, pacientes, especialidades, convênios
3	Sprint 2 — Admin Perfis	Usuários, perfis, JWT, segurança, relatórios
4	Sprint 3 — Agendamento Base	Agendar, remarcar, cancelar consulta
5	Sprint 4 — Agendamento Avançado	Disponibilidade, bloqueio, consultar agenda
6	Sprint 5 — Atendimento	Prontuário, relatório clínico, encaminhamento, urgência
7	Sprint 6 — Qualidade	Testes, Swagger, Postman, bug fixes
8	Sprint 7 — Entrega	Documentação, apresentação, release v1.0.0

Sprint 0 — Setup (Semana 1)

- [ ] Criar repositório GitHub
- [ ] Configurar `.gitignore` e branches (main, develop)
- [ ] Criar `docker-compose.yml` com 3 MySQL + rede
- [ ] Criar módulo `commons` (exceptions, DTOs, configs)
- [ ] Criar estrutura base dos 3 microsserviços (Spring Initializr)
- [ ] Configurar `pom.xml` e `application.yml` de cada serviço
- [ ] Testar conexão de cada serviço com seu banco
- [ ] Criar `build-and-run.sh`
- [ ] Escrever `README.md` inicial

Sprint 1 — Admin Base (Semana 2)

- [ ] Criar entities: Medico, Paciente, Especialidade, Convenio
- [ ] Criar repositories, DTOs, mappers
- [ ] Criar services com regras de negócio
- [ ] Criar controllers REST
- [ ] Validações (CPF, CNPJ, Bean Validation)
- [ ] Soft delete, paginação
- [ ] Swagger annotations
- [ ] Testes unitários

Sprint 2 — Admin Perfis + Segurança (Semana 3)

- [ ] Criar entities: Usuario, Perfil
- [ ] Implementar JWT (geração, validação)
- [ ] Spring Security + RBAC (ADM, MEDICO, SECRETARIA)
- [ ] Endpoints de login/registro
- [ ] BCrypt para senhas
- [ ] Relatórios gerenciais
- [ ] Testes

Sprint 3 — Agendamento Base (Semana 4)

- [ ] Criar entities: Consulta, Cancelamento
- [ ] Criar Feign Client para admin-service
- [ ] Agendar consulta (com validação de convênio via Feign)
- [ ] Remarcar consulta
- [ ] Cancelar consulta com motivo obrigatório
- [ ] Validação de conflito de horário
- [ ] Máquina de estados
- [ ] Testes

Sprint 4 — Agendamento Avançado (Semana 5)

- [ ] Criar entities: DisponibilidadeMedico, BloqueioAgenda
- [ ] Definir Disponibilidade (Médico)
- [ ] Bloquear Agenda (Secretária e Médico)
- [ ] Consultar Agenda com filtros
- [ ] Buscar horários disponíveis
- [ ] Validar disponibilidade e bloqueios ao agendar
- [ ] Testes

Sprint 5 — Atendimento (Semana 6)

- [ ] Criar entities: Prontuario, Atendimento, RelatorioClinico, Receita, SolicitacaoExame, Encaminhamento
- [ ] Feign Clients (admin + agendamento)
- [ ] Criar prontuário automático
- [ ] Registrar atendimento com escopo médico
- [ ] Criar/editar relatório clínico
- [ ] Definir urgência
- [ ] Atribuir a outro médico (encaminhamento)
- [ ] Agendar reconsulta via Feign
- [ ] Testes

Sprint 6 — Qualidade (Semana 7)

- [ ] Completar cobertura de testes (mínimo 70%)
- [ ] Swagger completo em todos os endpoints
- [ ] Postman collections completas
- [ ] Bug fixes, refatoração
- [ ] Testes ponta a ponta
- [ ] Revisão de logs e exceções

Sprint 7 — Entrega (Semana 8)

- [ ] README.md final, docs de arquitetura
- [ ] Apresentação
- [ ] Code review final
- [ ] Tag release v1.0.0
- [ ] Merge em main

---

8. DIVISÃO DE EQUIPE (5 pessoas)

Papel	Responsabilidades	Quem
Tech Lead	Arquitetura, code review, merge PRs, Docker	Membro 1
Backend Dev 1	`admin-service` + `commons`	Membro 2

Papel	Responsabilidades	Quem
Backend Dev 2	`admin-service` (perfis, JWT, relatórios)	Membro 3
Backend Dev 3	`agendamento-service` completo	Membro 4
Backend Dev 4 / QA	`atendimento-service` + testes + Postman	Membro 5

---

9. LISTA COMPLETA DE TAREFAS TÉCNICAS

commons (12 tarefas)

1. Criar módulo Maven
2. `RespostaErro.java`
3. `RespostaPaginada.java`
4. `ErroValidacao.java`
5. `RegraDeNegocioException.java`
6. `RecursoNaoEncontradoException.java`
7. `RecursoDuplicadoException.java`
8. `IntegracaoException.java`
9. `TratadorGlobalExcecoes.java`
10. `ValidadorCpf.java`
11. `ValidadorCnpj.java`
12. `ConstantesApi.java`

admin-service (32 tarefas)

1–4. Entities: Medico, Paciente, Especialidade, Convenio
5–8. Repositories
9–12. DTOs Requisicao
13–16. DTOs Resposta
17–20. Mappers
21–24. Services com regras
25–28. Controllers REST
29. Entity Usuario + Perfil
30. JwtService + SecurityConfig
31. RelatorioService + RelatorioController
32. Swagger annotations

agendamento-service (20 tarefas)

1–2. Entities: Consulta, Cancelamento
3–4. Entities: DisponibilidadeMedico, BloqueioAgenda
5–6. Enums: StatusConsulta, TipoConsulta
7–8. Repositories
9–12. DTOs

13–14. Mappers
15. Feign Client AdminServiceClient
16. ConsultaService (agendar, remarcar, cancelar)
17. DisponibilidadeService
18. BloqueioAgendaService
19. Controllers
20. Swagger annotations

atendimento-service (24 tarefas)

1–6. Entities: Prontuario, Atendimento, RelatorioClinico, Receita, SolicitacaoExame, Encaminhamento
7–8. Enums: NivelUrgencia, StatusEncaminhamento
9–10. Repositories
11–14. DTOs
15–16. Mappers
17–18. Feign Clients (Admin + Agendamento)
19. ProntuarioService
20. AtendimentoService
21. RelatorioClinicoService
22. EncaminhamentoService
23. Controllers
24. Swagger annotations

Infraestrutura (10 tarefas)

1. docker-compose.yml com 3 MySQL
2. Dockerfile de cada serviço
3. build-and-run.sh
4. .gitignore
5. README.md
6. docs/arquitetura.md
   7–9. Postman collections (1 por serviço)
7. Tag release v1.0.0

Total: ~98 tarefas

Continua na Parte 4 → Comunicação Feign, Segurança JWT, Exceções e Logs

04_comunicacao_seguranca

🏥 PARTE 4 — COMUNICAÇÃO, SEGURANÇA, EXCEÇÕES E LOGS

---

10. COMUNICAÇÃO ENTRE MICROSSERVIÇOS (OpenFeign)

10.1 O Que É OpenFeign

OpenFeign é um **cliente HTTP declarativo**. Em vez de escrever `RestTemplate`, você define uma **interface Java** e o Feign gera a implementação.

Dependência Maven

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

Habilitar na Application

```
@SpringBootApplication
@EnableFeignClients
public class AgendamentoServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(AgendamentoServiceApplication.class, args);
    }
}
```

10.2 Exemplo — Feign Client no agendamento-service

```
package br.com.clinica.agendamento.client;

@FeignClient(
    name = "admin-service",
    url = "${servicos.admin-service.url}",
    fallback = AdminServiceClientFallback.class
)
public interface AdminServiceClient {

    @GetMapping("/api/v1/pacientes/{id}/validar-ativo")
    Boolean validarPacienteAtivo(@PathVariable("id") Long id);

    @GetMapping("/api/v1/medicos/{id}/validar-ativo")
    Boolean validarMedicoAtivo(@PathVariable("id") Long id);

    @GetMapping("/api/v1/convenios/{id}/validar-ativo")
    Boolean validarConvenioAtivo(@PathVariable("id") Long id);
```

```
    @GetMapping("/api/v1/pacientes/{id}")
    PacienteResposta buscarPacientePorId(@PathVariable("id") Long id);

    @GetMapping("/api/v1/medicos/{id}")
    MedicoResposta buscarMedicoPorId(@PathVariable("id") Long id);
}
```

Configuração no application.yml

```
servicos:
  admin-service:
    url: ${ADMIN_SERVICE_URL:http://localhost:8081}
  agendamento-service:
    url: ${AGENDAMENTO_SERVICE_URL:http://localhost:8082}
```

10.3 Uso no Service

```
@Service
@RequiredArgsConstructor
@Slf4j
public class ConsultaService {

    private final ConsultaRepository consultaRepository;
    private final AdminServiceClient adminServiceClient;

    public ConsultaResposta agendarConsulta(ConsultaRequisicao requisicao) {
        // 1. Validar paciente via Feign
        Boolean pacienteAtivo =
adminServiceClient.validarPacienteAtivo(requisicao.getPacienteId());
        if (pacienteAtivo == null || !pacienteAtivo) {
            throw new RegraDeNegocioException("Paciente não encontrado ou
inativo");
        }

        // 2. Validar médico via Feign
        Boolean medicoAtivo =
adminServiceClient.validarMedicoAtivo(requisicao.getMedicoId());
        if (medicoAtivo == null || !medicoAtivo) {
            throw new RegraDeNegocioException("Médico não encontrado ou inativo");
        }

        // 3. Validar convênio (se paciente possui)
        PacienteResposta paciente =
adminServiceClient.buscarPacientePorId(requisicao.getPacienteId());
        if (paciente.isPossuiConvenio()) {
            Boolean convenioAtivo =
```

```
adminServiceClient.validarConvenioAtivo(paciente.getConvenioId());
            if (convenioAtivo == null || !convenioAtivo) {
                throw new RegraDeNegocioException("Convênio do paciente está
inativo ou inválido");
            }
        }

        // 4. Validar conflito de horário
        validarConflitoHorario(requisicao.getMedicoId(),
requisicao.getDataHora());

        // 5. Criar consulta
        Consulta consulta = ConsultaMapper.paraEntidade(requisicao);
        consulta.setStatus(StatusConsulta.AGENDADA);
        consulta = consultaRepository.save(consulta);

        log.info("Consulta agendada com sucesso: ID={}", consulta.getId());
        return ConsultaMapper.paraResposta(consulta);
    }
}
```

10.4 Timeouts e Retry

```yaml
spring:
  cloud:
    openfeign:
      client:
        config:
          default:
            connect-timeout: 5000
            read-timeout: 10000
            logger-level: FULL
```

10.5 Fallback

```
@Component
public class AdminServiceClientFallback implements AdminServiceClient {
    @Override
    public Boolean validarPacienteAtivo(Long id) { return false; }
    @Override
    public Boolean validarMedicoAtivo(Long id) { return false; }
    @Override
    public Boolean validarConvenioAtivo(Long id) { return false; }
    @Override
    public PacienteResposta buscarPacientePorId(Long id) { return null; }
    @Override
```

```
    public MedicoResposta buscarMedicoPorId(Long id) { return null; }
}
```

10.6 ErrorDecoder

```
@Component
public class DecodificadorErroFeign implements ErrorDecoder {
    @Override
    public Exception decode(String methodKey, Response response) {
        return switch (response.status()) {
            case 404 -> new RecursoNaoEncontradoException("Recurso não encontrado
no serviço remoto");
            case 400 -> new RegraDeNegocioException("Requisição inválida para o
serviço remoto");
            case 503 -> new IntegracaoException("Serviço temporariamente
indisponível");
            default -> new IntegracaoException("Erro na comunicação: " +
response.status());
        };
    }
}
```

---

11. AUTENTICAÇÃO E SEGURANÇA (JWT)

11.1 Fluxo JWT

```
1. Cliente → POST /api/v1/auth/login (usuario + senha)
2. admin-service valida credenciais → gera JWT
3. Cliente armazena token
4. Próximas requisições: Authorization: Bearer <token>
5. Cada serviço valida o token antes de processar
```

11.2 Roles Baseadas nos Atores UML

Role (Perfil)	Permissões
`ADM`	Tudo — CRUD completo, relatórios, gerenciar perfis
`MEDICO`	Agenda própria, atendimentos, relatórios clínicos, encaminhamentos
`SECRETARIA`	Agendar/remarcar/cancelar consultas, cadastrar pacientes, consultar agenda

11.3 JwtService

```java
@Service
@RequiredArgsConstructor
public class JwtService {

    @Value("${jwt.chave-secreta}")
    private String chaveSecreta;

    @Value("${jwt.expiracao}")
    private long expiracao; // 86400000 (24h)

    public String gerarToken(Usuario usuario) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("usuarioId", usuario.getId());
        claims.put("perfil", usuario.getPerfil().getNome());
        return construirToken(claims, usuario.getNomeUsuario(), expiracao);
    }

    private String construirToken(Map<String, Object> claims, String subject, long
expiracao) {
        return Jwts.builder()
                .setClaims(claims)
                .setSubject(subject)
                .setIssuedAt(new Date())
                .setExpiration(new Date(System.currentTimeMillis() + expiracao))
                .signWith(obterChaveAssinatura(), SignatureAlgorithm.HS256)
                .compact();
    }

    public boolean isTokenValido(String token) {
        try {

Jwts.parserBuilder().setSigningKey(obterChaveAssinatura()).build().parseClaimsJws(
token);
            return true;
        } catch (JwtException e) { return false; }
    }

    public String extrairNomeUsuario(String token) {
        return extrairClaim(token, Claims::getSubject);
    }

    public String extrairPerfil(String token) {
        return extrairClaim(token, claims -> claims.get("perfil", String.class));
    }

    private <T> T extrairClaim(String token, Function<Claims, T> resolver) {
        Claims claims = Jwts.parserBuilder().setSigningKey(obterChaveAssinatura())
```

```
                .build().parseClaimsJws(token).getBody();
        return resolver.apply(claims);
    }

    private Key obterChaveAssinatura() {
        return Keys.hmacShaKeyFor(Decoders.BASE64.decode(chaveSecreta));
    }
}
```

11.4 Security Config

```
@Configuration
@EnableWebSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtFiltroAutenticacao jwtFiltro;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(s ->
s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/v1/auth/login").permitAll()
                .requestMatchers("/swagger-ui/**", "/api-docs/**").permitAll()
                .requestMatchers(HttpMethod.DELETE, "/api/v1/**").hasRole("ADM")
                .requestMatchers("/api/v1/usuarios/**").hasRole("ADM")
                .requestMatchers("/api/v1/relatorios/**").hasRole("ADM")
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtFiltro,
UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(12);
    }
}
```

11.5 Filtro JWT

```java
@Component
@RequiredArgsConstructor
public class JwtFiltroAutenticacao extends OncePerRequestFilter {

    private final JwtService jwtService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
            HttpServletResponse response, FilterChain chain) throws
ServletException, IOException {
        String authHeader = request.getHeader("Authorization");
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            chain.doFilter(request, response);
            return;
        }
        String token = authHeader.substring(7);
        if (jwtService.isTokenValido(token)) {
            String usuario = jwtService.extrairNomeUsuario(token);
            String perfil = jwtService.extrairPerfil(token);
            UsernamePasswordAuthenticationToken auth = new
UsernamePasswordAuthenticationToken(
                usuario, null, List.of(new SimpleGrantedAuthority("ROLE_" +
perfil)));
            SecurityContextHolder.getContext().setAuthentication(auth);
        }
        chain.doFilter(request, response);
    }
}
```

12. TRATAMENTO GLOBAL DE EXCEÇÕES

12.1 Padrão de Resposta de Erro

```java
// commons/dto/RespostaErro.java
@Data @Builder @AllArgsConstructor @NoArgsConstructor
public class RespostaErro {
    private int status;
    private String erro;
    private String mensagem;
    private String caminho;
    private LocalDateTime timestamp;
    private List<ErroValidacao> errosValidacao;
}
```

```
@Data @AllArgsConstructor
public class ErroValidacao {
    private String campo;
    private String mensagem;
}
```

Exemplo de resposta:

```
{
  "status": 404,
  "erro": "Não Encontrado",
  "mensagem": "Paciente com ID 99 não encontrado",
  "caminho": "/api/v1/pacientes/99",
  "timestamp": "2026-05-09T10:30:00",
  "errosValidacao": null
}
```

12.2 Exceções Customizadas

```
public class RecursoNaoEncontradoException extends RuntimeException {
    public RecursoNaoEncontradoException(String msg) { super(msg); }
}
public class RecursoDuplicadoException extends RuntimeException {
    public RecursoDuplicadoException(String msg) { super(msg); }
}
public class RegraDeNegocioException extends RuntimeException {
    public RegraDeNegocioException(String msg) { super(msg); }
}
public class IntegracaoException extends RuntimeException {
    public IntegracaoException(String msg) { super(msg); }
}
```

12.3 Handler Global

```
@RestControllerAdvice
@Slf4j
public class TratadorGlobalExcecoes {

    @ExceptionHandler(RecursoNaoEncontradoException.class)
    public ResponseEntity<RespostaErro>
tratarNaoEncontrado(RecursoNaoEncontradoException ex, HttpServletRequest req) {
        log.warn("Recurso não encontrado: {}", ex.getMessage());
        return construirResposta(HttpStatus.NOT_FOUND, ex.getMessage(), req);
    }
```

```
    @ExceptionHandler(RecursoDuplicadoException.class)
    public ResponseEntity<RespostaErro> tratarDuplicado(RecursoDuplicadoException
ex, HttpServletRequest req) {
        return construirResposta(HttpStatus.CONFLICT, ex.getMessage(), req);
    }

    @ExceptionHandler(RegraDeNegocioException.class)
    public ResponseEntity<RespostaErro> tratarNegocio(RegraDeNegocioException ex,
HttpServletRequest req) {
        return construirResposta(HttpStatus.UNPROCESSABLE_ENTITY, ex.getMessage(),
req);
    }

    @ExceptionHandler(IntegracaoException.class)
    public ResponseEntity<RespostaErro> tratarIntegracao(IntegracaoException ex,
HttpServletRequest req) {
        log.error("Falha de integração: {}", ex.getMessage());
        return construirResposta(HttpStatus.SERVICE_UNAVAILABLE, ex.getMessage(),
req);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<RespostaErro>
tratarValidacao(MethodArgumentNotValidException ex, HttpServletRequest req) {
        List<ErroValidacao> erros =
ex.getBindingResult().getFieldErrors().stream()
                .map(fe -> new ErroValidacao(fe.getField(),
fe.getDefaultMessage())).toList();
        RespostaErro resposta = RespostaErro.builder()
                .status(400).erro("Requisição Inválida").mensagem("Erro de
validação")

.caminho(req.getRequestURI()).timestamp(LocalDateTime.now()).errosValidacao(erros)
.build();
        return ResponseEntity.badRequest().body(resposta);
    }

    private ResponseEntity<RespostaErro> construirResposta(HttpStatus status,
String msg, HttpServletRequest req) {
        RespostaErro resp = RespostaErro.builder()

.status(status.value()).erro(status.getReasonPhrase()).mensagem(msg)

.caminho(req.getRequestURI()).timestamp(LocalDateTime.now()).build();
        return new ResponseEntity<>(resp, status);
    }
}
```

12.4 Mapa de Códigos HTTP

Código	Quando	Exceção
`200`	Sucesso (GET, PUT)	—
`201`	Criado (POST)	—
`204`	Sem conteúdo (DELETE)	—
`400`	Dados inválidos	`MethodArgumentNotValidException`
`404`	Não encontrado	`RecursoNaoEncontradoException`
`409`	Duplicado (CPF)	`RecursoDuplicadoException`
`422`	Regra violada	`RegraDeNegocioException`
`503`	Serviço indisponível	`IntegracaoException`

---

13. LOGS E OBSERVABILIDADE

13.1 Logbook — Logging HTTP Automático

```
<dependency>
    <groupId>org.zalando</groupId>
    <artifactId>logbook-spring-boot-starter</artifactId>
    <version>3.7.0</version>
</dependency>
```

```
logbook:
  filter.enabled: true
  format.style: json
  obfuscate:
    headers: [Authorization]
    parameters: [senha]
```

13.2 Correlation ID (Rastreamento entre serviços)

```
@Component
public class FiltroCorrelationId extends OncePerRequestFilter {
    private static final String HEADER = "X-Correlation-ID";

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse
res, FilterChain chain)
            throws ServletException, IOException {
        String correlationId = req.getHeader(HEADER);
```

```
        if (correlationId == null || correlationId.isBlank()) {
            correlationId = UUID.randomUUID().toString();
        }
        MDC.put("correlationId", correlationId);
        res.setHeader(HEADER, correlationId);
        try { chain.doFilter(req, res); } finally { MDC.remove("correlationId"); }
    }
}
```

Propagar nas chamadas Feign:

```
@Component
public class InterceptadorFeignCorrelation implements RequestInterceptor {
    @Override
    public void apply(RequestTemplate template) {
        String id = MDC.get("correlationId");
        if (id != null) template.header("X-Correlation-ID", id);
    }
}
```

Formato do log:

```
logging:
  pattern:
    console: "%d{yyyy-MM-dd HH:mm:ss} [%thread] [%X{correlationId}] %-5level
%logger{36} - %msg%n"
```

13.3 Boas Práticas de Log

Nível	Quando	Exemplo
`ERROR`	Falha que impede operação	`log.error("Falha ao salvar", ex)`
`WARN`	Inesperado mas tratável	`log.warn("Paciente ID={} não encontrado", id)`
`INFO`	Operação de negócio	`log.info("Consulta agendada: ID={}", id)`
`DEBUG`	Detalhes técnicos	`log.debug("Validando CPF: {}", cpf)`

Regras: NUNCA logar senhas/tokens. SEMPRE usar `@Slf4j` (Lombok). NUNCA `System.out.println`.

Continua na Parte 5 → Testes, Swagger, Docker e Pipeline

05_testes_docker

🏥 PARTE 5 — TESTES, SWAGGER, DOCKER E PIPELINE

---

14. TESTES AUTOMATIZADOS

14.1 Pirâmide de Testes

```
        /\
       /  \        Testes E2E (Postman)
      /    \       — Poucos, caros
     /──────\
    /        \     Testes de Integração
   /          \    — Controller + banco H2
  /────────────\
 /              \  Testes Unitários
/                \ — Rápidos, muitos
/──────────────────\
```

14.2 Dependências

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>test</scope>
</dependency>
```

14.3 Teste Unitário — Service

```
@ExtendWith(MockitoExtension.class)
class MedicoServiceTest {

    @Mock
    private MedicoRepository medicoRepository;

    @Mock
    private EspecialidadeRepository especialidadeRepository;
```

```
    @InjectMocks
    private MedicoService medicoService;

    @Test
    @DisplayName("Deve criar médico com sucesso")
    void deveCriarMedicoComSucesso() {
        // Arrange
        MedicoRequisicao requisicao = MedicoRequisicao.builder()
                .nomeCompleto("Dra. Maria Souza")
                .crm("CRM/SP 12345")
                .especialidadeId(1L)
                .build();

        Especialidade especialidade =
Especialidade.builder().id(1L).descricao("Cardiologia").build();
        Medico medicoSalvo = Medico.builder().id(3L).nomeCompleto("Dra. Maria
Souza").crm("CRM/SP 12345").build();

        when(medicoRepository.existsByCrm(requisicao.getCrm())).thenReturn(false);

when(especialidadeRepository.findById(1L)).thenReturn(Optional.of(especialidade));
        when(medicoRepository.save(any(Medico.class))).thenReturn(medicoSalvo);

        // Act
        MedicoResposta resposta = medicoService.criar(requisicao);

        // Assert
        assertNotNull(resposta);
        assertEquals(3L, resposta.getId());
        assertEquals("Dra. Maria Souza", resposta.getNomeCompleto());
        verify(medicoRepository, times(1)).save(any(Medico.class));
    }

    @Test
    @DisplayName("Deve lançar exceção ao criar médico com CRM duplicado")
    void deveLancarExcecaoQuandoCrmDuplicado() {
        MedicoRequisicao requisicao = MedicoRequisicao.builder().crm("CRM/SP
12345").build();
        when(medicoRepository.existsByCrm(requisicao.getCrm())).thenReturn(true);

        assertThrows(RecursoDuplicadoException.class, () ->
medicoService.criar(requisicao));
        verify(medicoRepository, never()).save(any());
    }

    @Test
    @DisplayName("Deve lançar exceção ao buscar médico inexistente")
    void deveLancarExcecaoQuandoMedicoNaoEncontrado() {
```

```
        when(medicoRepository.findById(99L)).thenReturn(Optional.empty());
        assertThrows(RecursoNaoEncontradoException.class, () ->
medicoService.buscarPorId(99L));
    }
}
```

14.4 Teste de Integração — Controller

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureMockMvc
@ActiveProfiles("test")
class MedicoControllerIntegrationTest {

    @Autowired private MockMvc mockMvc;
    @Autowired private ObjectMapper objectMapper;
    @Autowired private MedicoRepository medicoRepository;

    @BeforeEach
    void setUp() { medicoRepository.deleteAll(); }

    @Test
    @DisplayName("POST /api/v1/medicos - Deve criar e retornar 201")
    void deveCriarMedicoERetornar201() throws Exception {
        MedicoRequisicao req = MedicoRequisicao.builder()
                .nomeCompleto("Dr. João").crm("CRM/SP
99999").especialidadeId(1L).build();

        mockMvc.perform(post("/api/v1/medicos")
                        .contentType(MediaType.APPLICATION_JSON)
                        .content(objectMapper.writeValueAsString(req)))
                .andExpect(status().isCreated())
                .andExpect(jsonPath("$.id").exists())
                .andExpect(jsonPath("$.nomeCompleto").value("Dr. João"));
    }

    @Test
    @DisplayName("GET /api/v1/medicos/{id} - Deve retornar 404")
    void deveRetornar404ParaIdInexistente() throws Exception {
        mockMvc.perform(get("/api/v1/medicos/999"))
                .andExpect(status().isNotFound())
                .andExpect(jsonPath("$.mensagem").exists());
    }
}
```

application-test.yml

```yaml
spring:
  datasource:
    url: jdbc:h2:mem:testdb
    driver-class-name: org.h2.Driver
    username: sa
    password:
  jpa:
    hibernate:
      ddl-auto: create-drop
    database-platform: org.hibernate.dialect.H2Dialect
```

14.5 Cobertura Recomendada

Camada	Mínimo	O Que Testar
Service	80%	Regras de negócio, validações, exceções
Controller	70%	Status codes, payloads
Repository	60%	Queries customizadas

---

15. TESTES MANUAIS (Postman)

15.1 Organização

```
📁 Clínica Médica
├── 📁 admin-service
│   ├── 📁 Médicos (POST, GET, PUT, DELETE)
│   ├── 📁 Pacientes
│   ├── 📁 Especialidades
│   ├── 📁 Convênios
│   ├── 📁 Usuários
│   └── 📁 Relatórios
├── 📁 agendamento-service
│   ├── 📁 Consultas (Agendar, Remarcar, Cancelar)
│   ├── 📁 Disponibilidade
│   └── 📁 Bloqueios
└── 📁 atendimento-service
    ├── 📁 Atendimentos
    ├── 📁 Relatórios Clínicos
    ├── 📁 Encaminhamentos
    └── 📁 Reconsultas
```

15.2 Cenários de Teste

Positivos (Happy Path)

- [ ] Criar recurso com campos válidos → 201
- [ ] Listar recursos → 200
- [ ] Buscar por ID existente → 200
- [ ] Atualizar existente → 200
- [ ] Deletar existente → 204
- [ ] Agendar consulta válida → 201
- [ ] Cancelar com motivo → 200

Negativos (Error Path)

- [ ] Criar sem campo obrigatório → 400
- [ ] CPF inválido → 400
- [ ] CPF duplicado → 409
- [ ] ID inexistente → 404
- [ ] Agendar em horário conflitante → 422
- [ ] Agendar em horário bloqueado → 422
- [ ] Cancelar sem motivo → 400
- [ ] Acessar sem token → 401
- [ ] Secretária tentar deletar → 403

---

16. DOCUMENTAÇÃO SWAGGER / OPENAPI

16.1 Dependência

```
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.3.0</version>
</dependency>
```

16.2 Configuração

```
@Configuration
public class SwaggerConfig {
    @Bean
    public OpenAPI customOpenAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("Admin Service API")
                .description("Microsserviço administrativo da clínica médica")
                .version("1.0.0"))
```

```
            .addSecurityItem(new SecurityRequirement().addList("Bearer Auth"))
            .components(new Components()
                .addSecuritySchemes("Bearer Auth",
                    new
SecurityScheme().type(SecurityScheme.Type.HTTP).scheme("bearer").bearerFormat("JWT
")));
    }
}
```

16.3 Anotações no Controller

```
@RestController
@RequestMapping("/api/v1/medicos")
@RequiredArgsConstructor
@Tag(name = "Médicos", description = "Gerenciamento de médicos da clínica")
public class MedicoController {

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    @Operation(summary = "Cadastrar médico", description = "Cadastra um novo
médico no sistema")
    @ApiResponses({
        @ApiResponse(responseCode = "201", description = "Médico criado com
sucesso"),
        @ApiResponse(responseCode = "400", description = "Dados inválidos"),
        @ApiResponse(responseCode = "409", description = "CRM já cadastrado")
    })
    public MedicoResposta criar(@RequestBody @Valid MedicoRequisicao requisicao) {
        return medicoService.criar(requisicao);
    }
}
```

Acesso: `http://localhost:8081/swagger-ui.html`

---

17. DOCKERIZAÇÃO

17.1 Dockerfile Padrão (multi-stage)

```
# Build
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests
```

```
# Runtime
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8081
ENTRYPOINT ["java", "-jar", "app.jar"]
```

17.2 Docker Compose

```
version: '3.8'
services:
  # ===== BANCOS =====
  admin-db:
    image: mysql:8.0
    container_name: admin-db
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: admin_db
    ports: ["3306:3306"]
    volumes: [admin_data:/var/lib/mysql]
    networks: [clinica-network]
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 5

  agendamento-db:
    image: mysql:8.0
    container_name: agendamento-db
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: agendamento_db
    ports: ["3307:3306"]
    volumes: [agendamento_data:/var/lib/mysql]
    networks: [clinica-network]
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 5

  atendimento-db:
    image: mysql:8.0
    container_name: atendimento-db
    environment:
```

```yaml
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: atendimento_db
    ports: ["3308:3306"]
    volumes: [atendimento_data:/var/lib/mysql]
    networks: [clinica-network]
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 5

  # ===== MICROSSERVIÇOS =====
  admin-service:
    build: ./admin-service
    container_name: admin-service
    ports: ["8081:8081"]
    environment:
      DB_HOST: admin-db
      DB_PORT: 3306
      DB_USERNAME: root
      DB_PASSWORD: root
    depends_on:
      admin-db: { condition: service_healthy }
    networks: [clinica-network]

  agendamento-service:
    build: ./agendamento-service
    container_name: agendamento-service
    ports: ["8082:8082"]
    environment:
      DB_HOST: agendamento-db
      DB_PORT: 3306
      DB_USERNAME: root
      DB_PASSWORD: root
      ADMIN_SERVICE_URL: http://admin-service:8081
    depends_on:
      agendamento-db: { condition: service_healthy }
      admin-service: { condition: service_started }
    networks: [clinica-network]

  atendimento-service:
    build: ./atendimento-service
    container_name: atendimento-service
    ports: ["8083:8083"]
    environment:
      DB_HOST: atendimento-db
      DB_PORT: 3306
      DB_USERNAME: root
```

```yaml
      DB_PASSWORD: root
      ADMIN_SERVICE_URL: http://admin-service:8081
      AGENDAMENTO_SERVICE_URL: http://agendamento-service:8082
    depends_on:
      atendimento-db: { condition: service_healthy }
      admin-service: { condition: service_started }
      agendamento-service: { condition: service_started }
    networks: [clinica-network]

volumes:
  admin_data:
  agendamento_data:
  atendimento_data:

networks:
  clinica-network:
    driver: bridge
```

17.3 Comandos Docker

```bash
docker-compose up -d --build          # Subir tudo
docker-compose up -d admin-db agendamento-db atendimento-db  # Só bancos
docker-compose logs -f admin-service  # Logs
docker-compose down                   # Parar
docker-compose down -v                # Parar + remover dados
```

---

18. PIPELINE DE BUILD E EXECUÇÃO

18.1 Maven Lifecycle

Comando	O que faz
`mvn clean`	Limpa target
`mvn compile`	Compila
`mvn test`	Roda testes
`mvn package`	Gera JAR
`mvn spring-boot:run`	Roda em dev

18.2 Script build-and-run.sh

```bash
#!/bin/bash
set -e
```

```
echo "========================================"
echo "   AURA CLINIC - Build & Run Pipeline"
echo "========================================"

echo "[1/4] Building commons..."
cd commons && mvn clean install -DskipTests && cd ..

echo "[2/4] Building admin-service..."
cd admin-service && mvn clean package -DskipTests && cd ..

echo "[3/4] Building agendamento-service..."
cd agendamento-service && mvn clean package -DskipTests && cd ..

echo "[4/4] Building atendimento-service..."
cd atendimento-service && mvn clean package -DskipTests && cd ..

echo "Starting Docker..."
docker-compose up -d --build

echo ""
echo "Serviços disponíveis:"
echo "  admin-service:       http://localhost:8081/swagger-ui.html"
echo "  agendamento-service: http://localhost:8082/swagger-ui.html"
echo "  atendimento-service: http://localhost:8083/swagger-ui.html"
```

Continua na Parte 6 → Versionamento, Checklists e Riscos

06_versionamento_checklists

📋 PARTE 6 — VERSIONAMENTO, CHECKLISTS E RISCOS

---

19. ESTRATÉGIA DE VERSIONAMENTO

19.1 GitFlow

```
main ────────●──────────────────────────●──────── (releases)
             │                          ▲
             ▼                          │ merge
develop ────●────●────●────●────●────●──────── (integração)
                 │         │    ▲
```

```
                ▼       ▼    |
         feature/A  feature/B
```

19.2 Workflow Passo a Passo

Criar Feature Branch

```
git checkout develop
git pull origin develop
git checkout -b feature/admin-crud-medicos
```

Trabalhar na Feature

```
git add .
git commit -m "feat(admin): criar entity Medico com anotações JPA"
git commit -m "feat(admin): implementar MedicoService com CRUD"
git commit -m "test(admin): adicionar testes unitários para MedicoService"
```

Criar Pull Request

```
git push origin feature/admin-crud-medicos
# Abrir PR no GitHub: feature/admin-crud-medicos → develop
```

Template de Pull Request

```
## Descrição
Implementação do CRUD de Médicos no admin-service.

## O que foi feito
- [x] Entity Medico com anotações JPA
- [x] MedicoRepository
- [x] MedicoService com validações (CRM único)
- [x] MedicoController (REST endpoints)
- [x] DTOs (Requisição/Resposta) e Mapper
- [x] Testes unitários (85% cobertura)
- [x] Swagger annotations

## Como testar
1. Subir admin-db: `docker-compose up admin-db`
2. Rodar: `mvn spring-boot:run`
3. Acessar: `http://localhost:8081/swagger-ui.html`

## Checklist
- [x] Código segue padrões do projeto
- [x] Testes passando
```

```
- [x] Sem System.out.println
- [x] Tratamento de exceções
```

Criar Release

```
git checkout develop && git pull origin develop
git checkout -b release/v1.0.0
# Ajustar versão nos pom.xml, testar
git commit -m "chore: preparar release v1.0.0"
git checkout main && git merge release/v1.0.0
git tag -a v1.0.0 -m "Release v1.0.0 - MVP Aura Clinic"
git push origin main --tags
git checkout develop && git merge release/v1.0.0
```

---

20. CHECKLISTS DE VALIDAÇÃO

20.1 Checklist Backend (por microsserviço)

- [ ] Entidades mapeadas com JPA (@Entity, @Table, @Column)
- [ ] Repositories criados com Spring Data JPA
- [ ] DTOs separados (Requisição/Resposta) — NUNCA expor Entity
- [ ] Mappers implementados (Entity ↔ DTO)
- [ ] Services com regras de negócio
- [ ] Controllers REST com endpoints padronizados
- [ ] Bean Validation (@NotBlank, @NotNull, @Valid)
- [ ] Soft delete (campo `ativo`)
- [ ] Paginação nos endpoints de listagem
- [ ] Lombok (@Data, @Builder, @RequiredArgsConstructor)
- [ ] Sem `System.out.println` — usar `@Slf4j`
- [ ] Sem lógica de negócio no Controller

20.2 Checklist Banco de Dados

- [ ] DDL de todas as tabelas
- [ ] Índices nos campos de busca frequente
- [ ] UNIQUE em CPF, CRM, CNPJ, nome_usuario
- [ ] Campos de auditoria (criado_em, atualizado_em)
- [ ] Dados iniciais (seeds)
- [ ] ENUM para campos com valores fixos
- [ ] FK constraints onde aplicável (mesmo banco)
- [ ] Nenhuma FK entre bancos de serviços diferentes

20.3 Checklist Endpoints

- [ ] Seguem `/api/v1/<recurso-plural>`
- [ ] Métodos HTTP corretos
- [ ] Status codes corretos (200, 201, 204, 400, 404, 409, 422)
- [ ] Respostas de erro padronizadas (RespostaErro)
- [ ] Paginação com page e size
- [ ] Testados no Postman (positivo + negativo)

20.4 Checklist Segurança

- [ ] JWT implementado
- [ ] Senhas com BCrypt
- [ ] Endpoints públicos definidos (login, swagger)
- [ ] Endpoints protegidos exigem Bearer token
- [ ] Roles configuradas (ADM, MEDICO, SECRETARIA)
- [ ] Dados sensíveis não expostos em logs

20.5 Checklist Docker

- [ ] Dockerfile funcional (multi-stage)
- [ ] Docker Compose com 3 bancos + 3 serviços
- [ ] Volumes para persistência MySQL
- [ ] Rede bridge entre containers
- [ ] Healthcheck nos bancos
- [ ] Variáveis de ambiente configuradas
- [ ] `docker-compose up -d --build` funciona

20.6 Checklist Testes

- [ ] Testes unitários dos Services (Mockito)
- [ ] Testes de integração dos Controllers (MockMvc)
- [ ] Cenários positivos e negativos
- [ ] application-test.yml com H2
- [ ] Mocks de Feign Clients nos testes
- [ ] Cobertura mínima 70%
- [ ] `mvn test` passa sem erros

20.7 Checklist Documentação

- [ ] README.md completo
- [ ] Swagger configurado em todos os serviços
- [ ] Endpoints documentados com @Operation
- [ ] Postman collections exportadas
- [ ] Instruções de setup no README

20.8 Checklist UML — Validação do Escopo

Módulo Administrativo

- [ ] ✅ Gerenciar Médicos
- [ ] ✅ Gerenciar Pacientes
- [ ] ✅ Gerenciar Especialidades
- [ ] ✅ Gerenciar Perfis (Usuários/permissões)
- [ ] ✅ Gerenciar Convênios
- [ ] ✅ Gerenciar Relatórios

Módulo de Agendamento

- [ ] ✅ Agendar Consulta (com Validar Convênio)
- [ ] ✅ Remarcar Consulta
- [ ] ✅ Cancelar Consulta
- [ ] ✅ Consultar Agenda
- [ ] ✅ Definir Disponibilidade
- [ ] ✅ Bloquear Agenda

Módulo de Atendimento

- [ ] ✅ Agendar Reconsulta
- [ ] ✅ Criar Relatório Clínico (com Escopo Médico + Consultar/Editar)
- [ ] ✅ Salvar Dados do Paciente
- [ ] ✅ Atribuir a Outro Médico
- [ ] ✅ Definir Urgência

---

21. RISCOS TÉCNICOS

21.1 Mapa de Riscos

#	Risco	Prob.	Impacto	Mitigação
1	**Serviço indisponível** — Feign falha	Alta	Alto	Fallback, retry, timeout
2	**Inconsistência de dados** — Paciente deletado mas consulta existe	Média	Alto	Soft delete; validar status via Feign
3	**Conflito de horário** — Duas requisições simultâneas	Média	Alto	UNIQUE INDEX (medico_id, data_hora); tratar constraint violation
4	**Bloqueio não verificado** — Agendamento em período bloqueado	Média	Alto	Validar bloqueios antes de agendar
5	**Acoplamento Feign** — Mudança na API quebra outro serviço	Alta	Médio	Versionamento (/v1/); contratos claros

#	Risco	Prob.	Impacto	Mitigação
6	**JWT secret fraco** — Token forjado	Média	Crítico	Secret 256 bits; variável de ambiente
7	**N+1 queries** — JPA carrega relações em loop	Alta	Médio	@EntityGraph ou JOIN FETCH
8	**Docker: portas em conflito**	Média	Baixo	Documentar portas; variáveis de ambiente
9	**Dados de teste em produção**	Baixa	Alto	Spring Profiles (dev/prod)
10	**Deadlock no banco**	Baixa	Alto	Transações curtas; HikariCP

21.2 Mitigações Detalhadas

Conflito de Horário (Race Condition)

```
@Transactional
public ConsultaResposta agendarConsulta(ConsultaRequisicao requisicao) {
    try {
        // ... validações ...
        return consultaRepository.save(consulta);
    } catch (DataIntegrityViolationException e) {
        throw new RegraDeNegocioException("Horário já ocupado para este médico");
    }
}
```

Bloqueio de Agenda

```
private void validarBloqueioAgenda(Long medicoId, LocalDateTime dataHora) {
    List<BloqueioAgenda> bloqueios = bloqueioRepository
        .findByMedicoIdAndDataInicioLessThanEqualAndDataFimGreaterThanEqual(
            medicoId, dataHora, dataHora);
    if (!bloqueios.isEmpty()) {
        throw new RegraDeNegocioException(
            "Médico possui bloqueio de agenda neste período: " +
bloqueios.get(0).getMotivo());
    }
}
```

Continua na Parte 7 → Escalabilidade, Melhorias Futuras e Boas Práticas

07_escalabilidade_praticas

🏥 PARTE 7 — ESCALABILIDADE, MELHORIAS FUTURAS E BOAS PRÁTICAS

22. ESCALABILIDADE E MELHORIAS FUTURAS

22.1 Roadmap de Evolução

```
v1.0 (Atual - MVP)          v2.0                      v3.0
──────────────────────  →  ──────────────────────  →  ──────────────────────
✅ REST Síncrono            📬 Mensageria (RabbitMQ)   🎡 Kubernetes
✅ OpenFeign                🗄️ Cache (Redis)           🔍 Service Discovery
✅ Docker Compose           📊 Monitoring              🚪 API Gateway
✅ JWT Manual               🔔 Notificações Email      📊 ELK Stack
✅ MySQL                    📧 Notification Service    🔄 CI/CD Pipeline
✅ 3 Microsserviços         📱 Frontend React/Angular  📱 App Mobile
```

22.2 Requisitos Opcionais — Como Encaixar no Futuro

Requisito Opcional 1: Cadastro de Funcionários Gerais

Impacto: Baixo — adicionar entity `Funcionario` no `admin-service`.

```
// Nova entity no admin-service
@Entity @Table(name = "funcionarios")
public class Funcionario {
    private Long id;
    private String nomeCompleto;
    private String rg;
    private String cpf;
    private String cargo; // RECEPCIONISTA, ENFERMEIRO, AUXILIAR, etc.
    private String ctps;
    private String pis;
    // ... endereço, contato, etc.
}
```

Requisito Opcional 2: Receituário e Exames Detalhados

Impacto: Médio — expandir entidades no `atendimento-service`.

```
// Expandir entity Receita
@Entity @Table(name = "receitas")
public class Receita {
    // ... campos existentes ...
```

```
    private String viaAdministracao; // ORAL, INTRAVENOSO, TOPICO
    private Boolean usoControlado;
    private String codigoMedicamento; // código ANVISA
    private LocalDate validadeReceita;
}
```

Requisito Opcional 3: Cancelamento com Auditoria

Impacto: Baixo — adicionar campos à entity `Cancelamento` no `agendamento-service`.

```
@Entity @Table(name = "cancelamentos")
public class Cancelamento {
    // ... campos existentes ...
    private String senhaSecretaria; // validada via BCrypt
    private String motivoDetalhado;
    private String ipRequisicao;
    private LocalDateTime dataHoraRegistro;
}
```

Regra adicional: Exigir senha da secretária no endpoint de cancelar e registrar em log de auditoria.

22.3 Melhorias Técnicas Detalhadas

Cache com Redis

```
Cenário: Listagem de especialidades consultada em todo agendamento.
Problema: Query no banco a cada requisição.
Solução: Cache Redis com TTL de 1 hora.

@Cacheable(value = "especialidades", key = "#id")
public EspecialidadeResposta buscarPorId(Long id) { ... }
```

Mensageria com RabbitMQ

```
Cenário: Ao cancelar consulta, notificar paciente.
Problema: Enviar email síncrono trava a requisição.
Solução: agendamento-service publica evento → notification-service consome.

agendamento-service → [RabbitMQ] → notification-service → Email/SMS
```

API Gateway (Spring Cloud Gateway)

```
Cenário: Cliente precisa saber URL de cada serviço.
Solução: Gateway centraliza em uma única URL.

Cliente → API Gateway (:8080) → admin-service
```

```
                                → agendamento-service
                                → atendimento-service
```

CI/CD com GitHub Actions

```yaml
# .github/workflows/ci.yml
name: CI Pipeline
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
      - run: cd commons && mvn clean install -DskipTests
      - run: cd admin-service && mvn clean verify
      - run: cd agendamento-service && mvn clean verify
      - run: cd atendimento-service && mvn clean verify
```

Monitoramento (Prometheus + Grafana)

```
Spring Boot Actuator → Prometheus → Grafana
Métricas: CPU, memória, requests/s, latência, erros, conexões DB
```

---

23. BOAS PRÁTICAS OBRIGATÓRIAS

23.1 Código

Regra	Por Quê
NUNCA expor Entity na API	Entity tem relações JPA → loop infinito e exposição de dados
SEMPRE usar DTOs	Requisição e Resposta separados dão flexibilidade
NUNCA lógica no Controller	Controller é porta de entrada — delega para Service
SEMPRE tratar exceções	Erros sem tratamento expõem stack trace
NUNCA `System.out.println`	Usar `@Slf4j` do Lombok
SEMPRE validar input	`@Valid` + Bean Validation previnem dados corrompidos
NUNCA hardcodar configs	Usar `application.yml` + variáveis de ambiente
SEMPRE soft delete	DELETE físico perde dados e quebra integridade
NUNCA retornar senha	Mesmo criptografada, é risco de segurança

Regra	Por Quê
SEMPRE paginar listagens	100 mil registros sem paginação trava a API

23.2 Arquitetura

Regra	Por Quê
NUNCA compartilhar banco	Acoplamento viola independência dos microsserviços
SEMPRE versionar APIs (`/v1/`)	Evolução sem quebrar consumidores
NUNCA Feign em loop	N chamadas HTTP = N * latência — usar batch endpoints
SEMPRE fallback para Feign	Serviço indisponível não deve derrubar quem consome
SEMPRE correlation-id	Sem ele, debugar fluxo entre 3 serviços é impossível

23.3 Ambientes (Spring Profiles)

Ambiente	Banco	Logs	Swagger	Debug
dev	MySQL local	DEBUG	Habilitado	Habilitado
test	H2 em memória	WARN	—	—
prod	MySQL em nuvem	WARN/ERROR	Desabilitado	Desabilitado

```yaml
# application-dev.yml
spring:
  jpa:
    show-sql: true
logging:
  level:
    br.com.clinica: DEBUG

# application-prod.yml
spring:
  jpa:
    show-sql: false
logging:
  level:
    br.com.clinica: WARN
springdoc:
  api-docs:
    enabled: false
```

REFERÊNCIA RÁPIDA DE COMANDOS

```
# Git
git checkout -b feature/nome-da-feature
git add . && git commit -m "feat(admin): mensagem"
git push origin feature/nome-da-feature

# Maven
mvn clean install          # Build completo
mvn spring-boot:run        # Rodar localmente
mvn test                   # Rodar testes

# Docker
docker-compose up -d       # Subir containers
docker-compose down        # Parar
docker-compose logs -f <servico>  # Ver logs

# MySQL (acessar banco no container)
docker exec -it admin-db mysql -u root -p
```

> 🎓 **NOTA FINAL:** Este guia simula um ambiente profissional de engenharia de software. Ao seguir todas as seções, os alunos terão experiência com as mesmas ferramentas, padrões e processos de empresas reais. O diferencial não está em "fazer funcionar", mas em fazer **como profissionais fazem** — com organização, padronização, testes, documentação e rastreabilidade.

Fim do Manual — 7 Partes Completas | Projeto Aura Clinic v1.0.0